

Clase 199 — Ingeniería de detección como disciplina

Parte: 8 — Blue Team, detección y SOC · Fuente: *The Sigma Specification* · MITRE ATT&CK · prácticas de Detection Engineering 🕒 Duración estimada: 110 min · Nivel: Avanzado

Objetivo

Tratar la detección como ingeniería, no como un conjunto de reglas sueltas: un ciclo de vida con requisitos, desarrollo, pruebas, despliegue, versionado y retiro, gestionado como código (detection-as-code) con CI/CD. Aprenderás a documentar detecciones, medir su calidad, validarlas contra ataques reales y evitar la degradación silenciosa de tu cobertura.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Aplicar** el ciclo de vida de una detección (idea → desarrollo → prueba → producción → retiro).
- 2. **Gestionar** detecciones como código en un repositorio con CI/CD.
- 3. **Documentar** cada detección con metadatos, contexto y respuesta esperada.
- 4. **Validar** detecciones con Atomic Red Team y pruebas automatizadas.
- 5. **Medir** calidad (precisión, falsos positivos, cobertura, deuda de detección).

Temas

#	Tema	Por qué importa
1	Detection Engineering como disciplina	De reglas ad hoc a proceso
2	Ciclo de vida de una detección	Estructura repetible y auditable
3	Detection-as-code y control de versiones	Trazabilidad y revisión
4	CI/CD para detecciones	Validación automática
5	Documentación y metadatos	Contexto para el analista
6	Validación con Atomic Red Team	Probar que realmente detecta
7	Calidad y deuda de detección	Evitar degradación silenciosa
8	Colaboración red↔blue (puente a purple)	Mejorar con adversario real

Definiciones y características

- **Ingeniería de detección:** disciplina de crear, probar y mantener detecciones con rigor de ingeniería. Característica: ciclo de vida, calidad medible y mejora continua.
- **Detection-as-code:** gestionar reglas como código versionado (Sigma en Git). Característica: revisión por pares, historial y reproducibilidad.

- **CI/CD de detecciones:** pipeline que valida sintaxis, convierte y prueba reglas automáticamente. Característica: previene reglas rotas en producción.
- **Metadatos de detección:** contexto de la regla (autor, ATT&CK, falsos positivos, respuesta). Característica: acelera el triaje y el mantenimiento.
- **Validación:** ejecutar la técnica y comprobar que la regla dispara. Característica: distingue cobertura real de teórica.
- **Deuda de detección:** reglas obsoletas, ruidosas o sin dueño. Característica: degrada la señal si no se gestiona.
- **Precisión de detección:** proporción de alertas verdaderas sobre el total. Característica: mide la calidad, no la cantidad.

Herramientas y preparación

- Un **repositorio Git** para tus reglas Sigma (clase 186) como fuente de verdad.
- **sigma-cli/pySigma** en un pipeline de CI (GitHub Actions/GitLab CI) para validar y convertir.
- **Atomic Red Team** para validar que cada detección dispara ante su técnica.
- Tu SIEM de laboratorio como destino de despliegue.
- Una plantilla de metadatos por detección.

Las validaciones con Atomic Red Team se ejecutan solo en tu laboratorio propio y aislado.

Laboratorio guiado — Un pipeline de detección como código

1. **Estructura el repo.** Crea `detections/` con tus reglas Sigma y una plantilla de metadatos (título, ATT&CK, fuente de datos, falsos positivos, respuesta).
2. **Añade validación de sintaxis.** Un job de CI que corra `sigma check` sobre todas las reglas al hacer push.
3. **Convierte automáticamente.** Otro paso que compile cada regla a tu backend (`sigma convert`) y falle si alguna no compila.
4. **Escribe una detección nueva.** Por ejemplo, ejecución de `certutil` descargando un archivo (T1105).
5. **Valida con Atomic.** Ejecuta el test atómico correspondiente en tu laboratorio y confirma que la regla dispara en el SIEM.
6. **Documenta la respuesta.** Añade a la regla qué debe hacer el analista cuando dispare (traje, contención).
7. **Gestiona el ciclo.** Marca una regla vieja y ruidosa para retiro; registra la decisión en el historial de Git.
8. **Mide.** Calcula la precisión de 3 detecciones con datos de tu laboratorio y anota tu deuda de detección.

Ejercicios

1. Diseña la plantilla de metadatos de una detección completa.
2. Escribe el pipeline de CI (pseudo-YAML) que valide y convierta reglas.
3. Valida una detección con el test de Atomic Red Team adecuado.
4. Define criterios objetivos para retirar una detección.
5. Calcula la precisión de una regla con TP/FP de ejemplo.
6. Explica cómo el versionado ayuda a auditar cambios de detección.

Reto verificable

Entrega un mini repositorio de detección como código con al menos tres reglas documentadas, un pipeline que valida y convierte sintaxis, y la evidencia de que una de ellas fue validada con Atomic Red Team en tu laboratorio. **Criterio de aceptación:** el pipeline falla ante una regla con sintaxis inválida y pasa con las correctas, cada detección incluye metadatos y respuesta esperada, y demuestras que la técnica ejecutada con Atomic dispara la regla correspondiente en el SIEM.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Regla rota llega a producción	Sin CI de validación; añade <code>sigma check</code> al pipeline
Nadie sabe qué hace una alerta	Falta documentación/respuesta; exige metadatos por regla
Cobertura "teórica" que no dispara	No validada; prueba con Atomic Red Team
Reglas ruidosas acumuladas	Deuda de detección sin gestionar; revisa y retira periódicamente
Cambios sin trazabilidad	Detecciones fuera de control de versiones; muévelas a Git

Preguntas frecuentes

¿Por qué tratar detecciones como software? Porque comparten problemas: calidad, regresiones, mantenimiento y colaboración. Aplicar control de versiones, revisión y CI evita que tu cobertura se degrade sin que nadie lo note.

¿Toda detección debe validarse con Atomic? Siempre que exista un test atómico aplicable, sí: la cobertura teórica engaña. Validar demuestra que la regla dispara ante la técnica real y no solo en el papel.

¿Cuándo retiro una detección? Cuando su ratio de falsos positivos es insostenible, la técnica ya no aplica, o otra regla la cubre mejor. Documenta la decisión: retirar también es ingeniería.

Referencias

- SigmaHQ y Sigma Specification — <https://github.com/SigmaHQ/sigma>
- Atomic Red Team — <https://github.com/redcanaryco/atomic-red-team>
- MITRE ATT&CK — <https://attack.mitre.org/>
- Palantir, "Alerting and Detection Strategy Framework" — <https://github.com/palantir/alerting-detection-strategy-framework>
- Splunk, "Detection Engineering" (recursos formativos).

Siguiendo clase

[Clase 200 - Purple team desde el lado defensivo](#)